FinSight

Regulatory-Grade Financial Intelligence System

Technical System Documentation

Generated Technical Reference

September 9, 2026

Abstract

FinSight is a multi-agent Retrieval-Augmented Generation (RAG) system for analyzing corporate financial filings (annual reports) with an explicit design mandate for *compliance-grade trustworthiness* rather than conversational convenience. Every claim surfaced to the user is traceable to a verbatim source citation (document, page, snippet); claims that cannot be verified against retrieved evidence are hard-blocked before reaching the output, not merely flagged. The system decomposes a user query into subtasks via a Planner agent, retrieves evidence through a hybrid dense/sparse retrieval pipeline, extracts candidate claims via an Analyst agent, verifies each claim against its source through an entailment-checking Auditor agent, performs deterministic cross-document comparison through a Comparator agent, and finally synthesizes a structured Markdown report through a Synthesizer agent. Every run produces a machine-readable audit log capturing the full decision trail. This document provides an exhaustive technical reference to the codebase: architecture, module-by-module responsibilities, data models, API surface, configuration, testing and evaluation methodology, and deployment topology.

Contents

List of Figures

List of Tables

Chapter 1

Introduction and Project Purpose

1.1 Overview

FinSight (version 1.0.0) analyzes public annual reports – in the shipped corpus, FY2024–FY2026 filings for three Indian IT services companies (Infosys, TCS, and Wipro) – and answers analyst-style natural language queries such as cross-company margin comparisons, revenue-driver analysis, risk-disclosure extraction, and capital expenditure guidance comparison.

The project is deliberately engineered against a written specification (`FinSight_Project_Specification.pdf`, at the repository root) and is framed in its own design records (`docs/DECISIONS.md`) as differentiated from a "standard GenAI portfolio project": rather than a single LLM call over retrieved context, it implements a six-stage agent pipeline with hard verification gates, deterministic arithmetic where correctness matters, and full auditability.

1.2 Design Mandate

Four requirements recur throughout the documentation and the code itself, and they motivate essentially every architectural decision described in this document:

1. **Traceability.** Every factual claim in a synthesized report must carry a citation: source document, page number, and verbatim snippet.
2. **Hard blocking, not soft flagging.** Claims that cannot be entailed by retrieved evidence are removed from the final report entirely – they are not shown to the user with a caveat. This is enforced by the Auditor agent (Section 5.1.4).
3. **Cross-document reasoning as a first-class capability.** Comparing figures across multiple filings (e.g., Infosys vs. TCS, FY2024 vs. FY2025) is handled by a dedicated Comparator agent that performs *no* LLM arithmetic – all currency/unit conversion is done deterministically in Python (Section 5.3.6) before the LLM ever compares figures.
4. **Full auditability.** Every query execution persists a structured JSON audit log (Section 6.4) recording the plan, every retrieval, every claim (verified, uncertain, or blocked), and every agent invoked, with end-to-end latency.

1.3 Confidence Model

Rather than a binary answerable/unanswerable decision, FinSight assigns each claim a composite confidence score (Section 5.4.1) and classifies it into one of three tiers:

Status	**Meaning**	**Surfaced to user?**
`VERIFIED`	Entailed by evidence, confidence $\geq$ threshold	Yes
`UNCERTAIN`	Weak support or below threshold	Yes, prefixed `[UNCERTAIN]`
`UNVERIFIABLE`	Absent, contradictory, or fabricated-event pattern	No (blocked)

Chapter 2

System Architecture

2.1 Orchestration Framework

FinSight is orchestrated by a **LangGraph** `StateGraph` – a single compiled graph, built once and reused as a process-wide singleton by `orchestration/graph.py::get_graph()`. This replaced a Semantic Kernel dispatch layer (documented as `docs/DECISIONS.md` Decision 11: the pipeline shape never varies with the query, only the fan-out width does, which is exactly what LangGraph's `Send` API expresses without a hand-rolled `asyncio.gather` and manual join).

The graph has five nodes:

- `plan` – calls the PlannerAgent (`agents/router.py::plan_task`).
- `retrieve_analyze` – one instance per subtask, dispatched via `Send()`, running RetrieverAgent then AnalystAgent for that subtask.
- `audit` – calls AuditorAgent over every extracted claim in one batch.
- `compare` – calls ComparatorAgent over the unit-normalized subtask results; runs in the same superstep as `audit`, since neither depends on the other.
- `synthesize` – calls SynthesizerAgent once both `audit` and `compare` have completed.

Retriever, Analyst, Auditor, and Comparator are plain `async` functions called directly from their nodes – there is no dispatch layer between a node and the agent function it calls. Planner and Synthesizer remain prompt-only roles: they call `core/groq_client.py::chat_completion()` / `chat_completion_hedged()` directly, which keeps those two calls on the retry/reserve/hedge path.

2.2 The Six-Stage Pipeline

1. **PlannerAgent** decomposes the user query into 2–6 subtasks (a JSON list of strings).
2. **RetrieverAgent** (run per subtask, in parallel): hybrid BM25 + dense retrieval, Reciprocal Rank Fusion, cross-encoder reranking, and confidence scoring, producing ranked chunks with citations.
3. **AnalystAgent** (per subtask): extracts KPIs and verbatim claims from the retrieved chunks via an LLM call constrained to forbid any derived/computed figure.

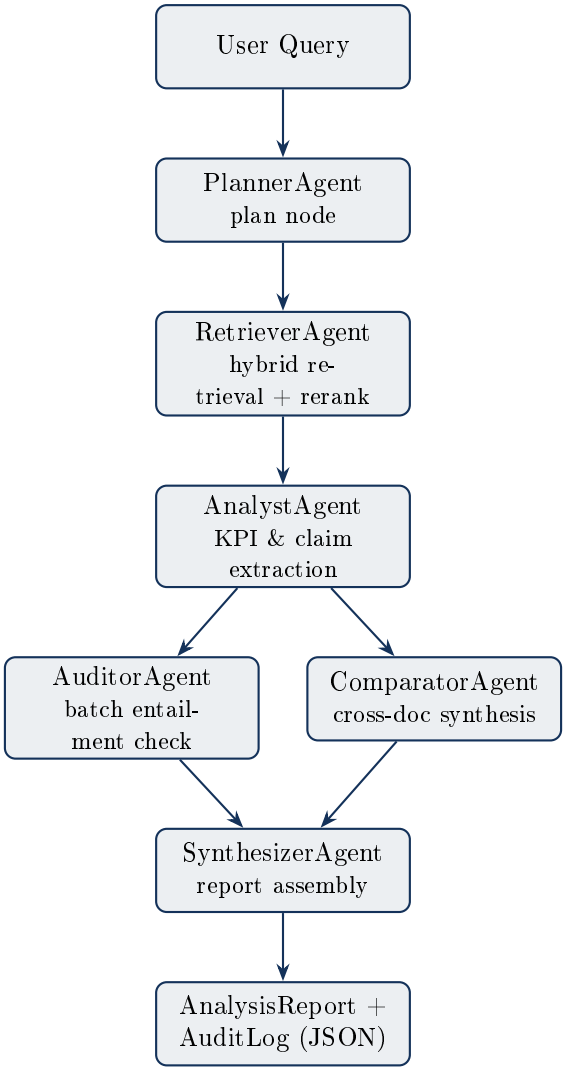

Figure 2.1: The six-stage agent pipeline, expressed as a LangGraph `StateGraph`. Auditor and Comparator run concurrently as one superstep on both the blocking and streaming routes, since neither depends on the other's output – only on the shared `subtask_results`.

4. **AuditorAgent**: a single batched LLM call verifies every claim produced by every subtask via entailment checking against its supporting snippet, classifying each as VERIFIED, UNCERTAIN, or UNVERIFIABLE.
5. **ComparatorAgent** (graph superstep, concurrent with the Auditor): performs cross-document comparison of pre-normalized figures, flagging deltas exceeding a 15% deviation threshold, without performing any arithmetic itself.
6. **SynthesizerAgent**: assembles a structured Markdown report from verified and uncertain claims plus the comparison output.

2.3 Context Propagation

Citations and claims are threaded through the pipeline as typed fields on `orchestration/graph.py::FinSightS` (a `TypedDict`), never serialized to JSON and re-parsed between stages. Fan-out fields (`subtask_results`, `errors`) use `Annotated[list[...], operator.add]` reducers so LangGraph merges the parallel `retrieve_analyze` branches automatically:

```
user_task → subtasks (list[str]) → subtask_results (reducer-merged) →
      verified/uncertain/unverifiable + comparison → final report
```

2.4 Concurrency and Latency Management

Both the blocking and streaming routes drive the same compiled graph through `orchestration/runner.py::run_`
which applies several latency-reduction strategies inherent to the graph's structure:

- All subtasks' Retriever + Analyst calls run in parallel via LangGraph's `Send` fan-out.
- Auditor and Comparator run concurrently as one superstep rather than sequentially, on *both* routes now (previously stream-only), since neither consumes the other's output.
- `core/groq_client.py::chat_completion_hedged()` implements *request hedging*: the primary LLM call is fired first; if it has not returned within a configurable `hedge_after` interval (8.0 s in the streaming route), the reserve endpoint is fired concurrently and whichever completes first is used. This bounds tail (p95) latency without doubling average cost.

2.5 Identity and Memory

Two additional best-effort nodes bracket the pipeline: `recall_memory` (before `plan`) and `remember` (after `synthesize`), backed by `memory/store.py::MemoryService`. A memory outage degrades to no recalled context / no write, never a failed query – see `docs/DECISIONS.md` Decision 12 for the full reasoning.

Identity. `api/middleware/auth.py::AuthMiddleware` (pure ASGI, header-only) resolves an `Authorization: Bearer <key>` header to a `user_id` via `settings.api_keys` (`"key:user_id,..."`). Unconfigured (the default), every request is `user_id="anonymous"` – local dev, the eval harness, and CI need no setup. This is identity for scoping memory, not an account system: no RBAC, one flat key → id mapping.

Storage. One new Qdrant collection, `finsight_memory`, holding `MemoryRecord` points: `memory_id, user_id, session_id, task_id, text, timestamp`. `retrieval/qdrant_store.py::QdrantStore` gained two additive constructor parameters (`collection`, `vector_size`) rather than a second client class, so every existing retrieval call site is unaffected. One record is written per completed turn – there is no separate LLM-based "consolidation" step, deliberately: a compliance system's memory should never be a new synthesis of what happened, only a record of it.

Recall draws on the same records two different ways:

- **Short-term** (this session): filter by `session_id`, rank by recency, limit `settings.session_max_turns` (6).
- **Long-term** (this user, across sessions): filter by `user_id`, rank by vector similarity to the current query, limit `settings.long_term_top_k` (3).

Where memory can and cannot reach. `memory/consolidate.py::format_memory_context` renders both into one text block, injected only into `PLANNER_PROMPT` (via `agents/router.py::plan_task`'s `memory_context` parameter). `SYNTHESIZER_PROMPT` is untouched. This is the load-bearing constraint: memory can bias what the Planner searches for, but it can never become a claim, because it never enters the claims/citations path the Auditor and Synthesizer operate on.

Transparency. `AuditLog` now carries `user_id` and `session_id`. `GET /sessions/{id}` and `GET /memory` let a caller inspect exactly what is recalled about them (their own records only – a 404, not silent filtering, on someone else's session); `DELETE` on both clears it.

Chapter 3

Repository Layout

```
FinSight/
|-- orchestration/              LangGraph StateGraph orchestration
|   |-- graph.py                 FinSightState, nodes, Send fan-out, compiled singleton
|   `-- runner.py                run_pipeline() - astream() driver, latency accounting
|-- memory/                     Per-user session + cross-session memory
|   |-- store.py                 MemoryService - session recall/write, long-term recall
|   `-- consolidate.py           build_turn_text(), format_memory_context() - no LLM call
|-- agents/                     Six-stage agent pipeline (graph nodes call these directly)
|   |-- router.py                PlannerAgent - plan_task()
|   |-- retriever.py             RetrieverAgent - RetrievalService, retrieve_chunks()
|   |-- analyst.py               AnalystAgent - analyze_chunks()
|   |-- comparator.py            ComparatorAgent - compare_results()
|   |-- auditor.py               AuditorAgent - audit_claims()
|   `-- synthesizer.py           synthesize_report() (direct LLM call)
|-- api/                        FastAPI application
|   |-- main.py                  App factory, lifespan, static mounting, /health
|   |-- metrics_store.py         In-process MetricsStore singleton
|   |-- middleware/auth.py       API key -> request.state.user_id (pure ASGI)
|   |-- middleware/guardrails.py  Prompt-injection detection (pure ASGI)
|   |-- routes/
|   |   |-- query.py             POST /query (blocking pipeline)
|   |   |-- query_stream.py      POST /query/stream (SSE pipeline)
|   |   |-- ingest.py            POST /ingest/upload
|   |   |-- eval.py              GET /eval/*
|   |   |-- sessions.py          GET/DELETE /sessions/{id}
|   |   |-- memory.py            GET/DELETE /memory
|   |   `-- metrics.py           GET /metrics/json
|   `-- static/
|       |-- index.html           Web UI (query interface + pipeline visualizer)
|       `-- dashboard.html       Metrics dashboard
|-- core/                       Shared infrastructure
|   |-- config.py                pydantic-settings Settings (env vars)
|   |-- models.py                Dataclasses & enums
|   |-- prompts.py               Planner/Synthesizer prompt templates
|   |-- groq_client.py           Async LLM client: retry, fallback, hedging
|   `-- unit_normalizer.py       Deterministic INR/USD normalizer
|-- retrieval/                  Hybrid retrieval pipeline
```

```
|   |-- qdrant_store.py          Async Qdrant client wrapper
|   |-- embedder.py              SentenceTransformer wrapper
|   |-- bm25.py                  BM25Okapi wrapper
|   |-- hybrid.py                Reciprocal Rank Fusion
|   |-- reranker.py              CrossEncoder wrapper
|   `-- confidence.py            5-signal composite confidence scoring
|-- ingestion/                   PDF -> Qdrant ingestion pipeline
|   |-- parser.py                PyMuPDF page/block/span extraction
|   |-- chunker.py               Heading-aware sliding-window chunker
|   |-- metadata.py              Section/fiscal-year/company detection
|   `-- pipeline.py              ingest_pdf(), ingest_directory()
|-- observability/tracer.py      structlog config, @traced decorator, OTel
    setup
|-- evaluation/
|   |-- harness.py               Async CLI test runner
|   |-- queries/                 happy_path.json, adversarial.json
|   `-- results/                 Timestamped harness_<epoch>.json results
|-- tests/                       pytest unit tests
|-- scripts/download_filings.py  Multi-strategy annual-report downloader
|-- data/filings/                Seed PDFs (Infosys / TCS / Wipro)
|-- audit_logs/                  Per-run JSON audit artifacts
|-- qdrant_storage/               Qdrant on-disk vector data
|-- docs/
|   |-- ARCHITECTURE.md          Architecture reference
|   `-- DECISIONS.md             Design-decision records
|-- .github/workflows/ci.yml     GitHub Actions CI (ruff + pytest)
|-- Dockerfile                    Multi-stage build
|-- docker-compose.yml            qdrant + api services
`-- pyproject.toml                Dependencies, ruff & pytest config
```

Two files, `retrieval/chunker.py` and `retrieval/dummy_test.py`, are development-era scratch duplicates of the canonical ingestion logic used for manual PDF experimentation; they are not part of the production import graph reached by `ingestion/pipeline.py`.

Chapter 4

Technology Stack

4.1 Core Dependencies

Python ≥3.11, managed with `uv`. Key libraries declared in `pyproject.toml`:

Category	Library	Role
PDF parsing	`pymupdf` (fitz)	Page/block/span text extraction
Vector database client	`qdrant-client`	Async Qdrant driver
Embeddings & reranking	`sentence-transformers`	Bi-encoder and cross-encoder models
Keyword search	`rank-bm25`	BM25Okapi sparse retrieval
Orchestration	`langgraph`	StateGraph orchestration: Send fan-out, typed state, custom streaming
LLM client	`openai`	OpenAI-compatible client pointed at Groq
LLM client (alt.)	`groq`	Native Groq SDK
API framework	`fastapi`	HTTP API
ASGI server	`uvicorn[standard]`	Application server
Configuration	`pydantic-settings`	Typed env-var settings
Logging	`structlog`	Structured logging
HTTP client	`httpx`	Async HTTP
Numerics	`numpy`	Vector math
Observability	`opentelemetry-*`	Tracing (OTLP/gRPC exporter)
Uploads	`python-multipart`	Multipart form parsing

Development dependencies: `pre-commit`, `ruff` (line-length 100, target py311, rule sets E/F/I), `pytest` + `pytest-asyncio` (`asyncio_mode="auto"`), `httpx` (for test client usage).

4.2 External Services

- **Qdrant** – vector database, collection `finsight_chunks`, 384-dimensional vectors (matching `all-MiniLM-L6-v2`), cosine distance.
- **Groq** – hosts the primary LLM, **Llama 3.3 70B** (`llama-3.3-70b-versatile`), via an OpenAI-compatible API. An optional reserve endpoint (any OpenAI-compatible provider) can be configured for hedging/failover.

4.3 Frontend

The UI (`api/static/index.html`, `dashboard.html`) is plain HTML/CSS/JS – no JavaScript framework – styled as a dark "compliance terminal" with a live pipeline visualizer that consumes the `/query/stream` SSE events, plus a separate metrics dashboard consuming `/metrics/json`.

Chapter 5

Module Reference

This chapter documents every module's responsibility, key functions, and inputs/outputs.

5.1 Agents (agents/)

5.1.1 router.py — PlannerAgent

```
async def plan_task(user_task: str, *, streaming: bool = False) ->
    list[str]:
     ...
```

Called once, from `orchestration/graph.py::plan_node`. Builds `PLANNER_PROMPT.format(user_task=...)` and calls `chat_completion()` (or `chat_completion_hedged()` when `streaming=True`, taken from graph state). Parses the LLM's JSON list output; on JSON-parse failure falls back to bullet/numbered-list regex parsing; if that also fails, falls back to the single-element list `[user_task]`.

5.1.2 retriever.py — RetrieverAgent

`RetrievalService` is a framework-agnostic class (no dispatch-layer decorators) owning the Qdrant handle and the process-lifetime BM25 corpus cache; the process-wide singleton is returned by `get_retrieval_service()` and used directly by the graph:

```
async def retrieve(self, subtask, company_filter="",
    fiscal_year_filter="") -> list[RankedChunk]:
```

It lazily builds a BM25 index once per process (guarded by an `asyncio.Lock` with double-checked initialization) over the full Qdrant corpus, obtained via `scroll_all()`. The real work is delegated to the module-level `retrieve_chunks()`, called from `orchestration/graph.py::retrieve_analyze_nod` which:

1. Filters BM25 payloads in-memory for company/fiscal-year (handling `"FY2024"` vs. `"2024"` via substring matching, since Qdrant's exact-match filter cannot).
2. Embeds the query.
3. Runs BM25 search and dense Qdrant search concurrently (`asyncio.gather`).
4. Fuses the two ranked lists via Reciprocal Rank Fusion.

5. Reranks the fused candidates via a cross-encoder.
6. Builds `RankedChunk` objects with composite confidence scores.

5.1.3 `analyst.py` — AnalystAgent

```
async def analyze_chunks(subtask, chunks_data) -> dict:  # {"kpis": [...], "claims": [...]}
```

Called directly from `orchestration/graph.py::retrieve_analyze_node` with the chunk list already deserialized – no JSON round-trip. It builds a context string from up to `settings.rerank_top_k` chunks and calls the LLM with a strict system prompt that **forbids extracting any derived or computed figure** – ratios, per-unit metrics, currency conversions, or cross-company combinations are disallowed; only verbatim printed figures may become claims. Confidence is assigned deterministically by section type (0.9 audited financials / 0.75 notes / 0.6 MD&A-or-letter). At most five claims are extracted per call. On JSON-parse failure, a regex-based fallback extraction is applied.

5.1.4 `auditor.py` — AuditorAgent

```
async def audit_claims(claims, chunks_by_subtask, confidence_threshold=None,
                       original_query="") -> tuple[list, list, list]:
```

Called directly from `orchestration/graph.py::audit_node`, which does *not* catch exceptions from this call – an auditor failure aborts the whole graph run rather than shipping a report with unaudited claims. Claims are split into two groups:

- Claims *with* a supporting snippet: batch-audited in a **single** LLM call (`_batch_entailment()`) for efficiency and consistency.
- Claims *without* a supporting snippet: instantly classified `UNVERIFIABLE` with reason `"No supporting snippet provided"` – no LLM call needed.

The batch-entailment prompt applies two checks, in order:

1. **Fabricated-event detection**: if the original query names a specific external company plus an action verb, and the claim's topic is unrelated, the claim is forced to `UNVERIFIABLE` regardless of superficial snippet overlap. This targets adversarial queries such as "what did the CEO say about acquiring Google."
2. **Standard entailment classification**: `VERIFIED` if the snippet entails the claim and confidence $\geq$ threshold; `UNCERTAIN` if support is weak or confidence is below threshold; `UNVERIFIABLE` if the snippet is absent or contradicts the claim.

Returns a triple `(verified, uncertain, unverifiable)` of `AuditedClaim` lists.

5.1.5 `comparator.py` — ComparatorAgent

```
async def compare_results(subtask_results, original_query) -> dict:
```

Called directly from `orchestration/graph.py::compare_node`, after `normalize_subtask_results()` has run; the node catches any exception from this call and degrades to an empty comparison rather than failing a run that has verified claims. It calls the LLM under a system prompt that

prohibits any arithmetic: no per-unit metrics, no currency/scale conversion, no percentage-delta computation, no cross-company arithmetic. The LLM's sole job is to place pre-normalized (already converted to Rs. crore) verbatim values side by side and flag anomalies – deviations exceeding 15% between comparable figures, unit/currency mismatches, or contradictions with auditor statements. Output:

```
{"deltas": [...], "cross_document_claims": [...], "summary": "..."}
```

5.1.6 synthesizer.py — SynthesizerAgent

```
def synthesize_report(query, verified_claims, uncertain_claims,
                      comparison, task_id) -> str:
```

Called from `orchestration/graph.py::synthesize_node`, which waits for both the `audit` and `compare` nodes – the graph's join, expressed as two incoming edges rather than a manual `asyncio.gather`. A direct `chat_completion_hedged()` call using the `SYNTHESIZER_PROMPT` template from `core/prompts.py`. Returns an "insufficient evidence" message outright if there are no verified or uncertain claims. The prompt enforces: only verbatim figures with inline citations; an `[UNCERTAIN]` prefix on uncertain claims; omission of derived figures unless pre-labeled `[converted from ...]`; no arithmetic; explicit `N/A` for missing data; and a fixed GitHub-Flavored-Markdown section structure (Executive Summary / Key Findings / Comparative Analysis / Risk Flags).

5.2 Memory (memory/)

5.2.1 store.py — MemoryService

```
async def recall_session(self, session_id, limit=None) -> list[
    MemoryRecord]:
async def recall_long_term(self, user_id, query, top_k=None) -> list[
    MemoryRecord]:
async def write(self, *, user_id, session_id, task_id, text) ->
    MemoryRecord:
```

Wraps a `QdrantStore` pointed at `settings.memory_collection` (`finsight_memory`), lazily `ensure_collection()`-ing on first use – the same lazy-init shape as `agents/retriever.py::RetrievalService`. `recall_session` is an exact `session_id` filter (`scroll_filtered`), sorted by `timestamp`, most recent `settings.session_max_turns`. `recall_long_term` is a `user_id`-filtered dense vector search on the embedding of the *current* query. Called directly from `orchestration/graph.py::recall_memory_` and `::remember_node` – no dispatch layer, same pattern as every other capability class in this codebase.

5.2.2 consolidate.py

```
def build_turn_text(query, subtasks, summary) -> str:
def format_memory_context(short_term, long_term) -> str:
```

Pure functions, no LLM call. `build_turn_text` is exactly `"Query: ... \nPlan: ... \nSummary: ..."` truncated to a 400-character summary excerpt – the memory record is what the pipeline already produced, never a new synthesis of it. `format_memory_context` renders recalled records into the block appended to `PLANNER_PROMPT`; empty when nothing was recalled.

5.3 Core Infrastructure (`core/`)

5.3.1 `config.py`

`Settings(BaseSettings)` – a `pydantic-settings` model reading `.env`. All fields are enumerated in Table 8.1.

5.3.2 `models.py`

The canonical dataclasses and enums shared across the entire system; see Chapter 6.

5.3.3 `prompts.py`

Contains the `PLANNER_PROMPT` and `SYNTHESIZER_PROMPT` string templates as plain `str.format` strings – no framework templating, no kernel or service object. Imported directly by `agents/router.py` and `agents/synthesizer.py`.

5.3.4 `orchestration/graph.py`

Builds and holds the compiled `StateGraph` singleton (`get_graph()`), constructed once by `build_graph()`. Defines `FinSightState` (the typed pipeline state), the five nodes, the `Send`-based fan-out (`fan_out()`), and `AGENT_SEQUENCE` (the fixed agent order recorded in the audit log, since the parallel audit/compare superstep makes node-completion order racy). `orchestration/runner.py` drives `graph.astream()` and derives per-agent latency from the stream's `updates` events.

5.3.5 `groq_client.py`

Lazily-initialized singleton `AsyncOpenAI` clients (primary, and optionally a reserve client selected when fallback environment variables are set).

- `chat_completion()`: retries on `RateLimitError` / HTTP 503 with backoff `[1.0, 2.0, 4.0]` seconds; breaks immediately to the reserve client on timeout or connection errors; raises `RuntimeError` if both primary and reserve fail.
- `chat_completion_hedged()`: implements the hedged dual-request pattern described in Section 2.2 – fires the primary request, and after a configurable delay fires the reserve concurrently, returning whichever completes first.

5.3.6 `unit_normalizer.py`

A pure-Python, regex-driven, deterministic currency/unit normalizer: `normalize_text()` and `normalize_subtask_results()`. Converts expressions such as `$X billion/million/...` and `Rs/INR X crore/lakh/million/...` into a canonical `Rs. X crore [converted from ...]` form using `Decimal` arithmetic (`USD_TO_INR = 84`). This module exists specifically so that the Comparator and Synthesizer LLM stages never perform currency or scale arithmetic themselves – all such conversion happens deterministically in Python before the LLM sees the data, eliminating a whole class of hallucinated arithmetic.

5.4 Retrieval Pipeline (`retrieval/`)

`qdrant_store.py` `QdrantStore` wraps `AsyncQdrantClient`: `ensure_collection()` (384-dim, cosine distance), `upsert_chunks()`, `dense_search()` (with optional payload `Filter`), `scroll_all()` (paginated full-corpus fetch feeding the BM25 index), `delete_by_doc_id()`, `collection_info()`.

`embedder.py` Lazy `SentenceTransformer(all-MiniLM-L6-v2)` singleton; `embed_query()`, `embed_texts()` (normalized embeddings).

`bm25.py` `BM25Retriever` wraps `rank_bm25.BM25Okapi`; a custom `_tokenize()` regex deliberately keeps `%` attached to its number (so `20.7%` tokenizes as one token) since financial figures with percent signs are common retrieval targets.

`hybrid.py` `reciprocal_rank_fusion(ranked_lists, k=60)` – a purely rank-based fusion that is invariant to the differing score scales of BM25 and cosine similarity.

`reranker.py` Lazy `CrossEncoder(ms-marco-MiniLM-L6-v2)` singleton; `rerank(query, candidates, top_k)` replaces stage-1 fused scores entirely with cross-encoder relevance scores.

`confidence.py` `compute_confidence()` and `build_ranked_chunks()`; see Section 5.4.1.

5.4.1 Confidence Scoring

`compute_confidence()` computes a weighted composite of five signals:

Signal	Weight	Description
Retrieval score	35%	Cross-encoder rerank score
Section-type weight	25%	Static weight by document section (see Table 5.1)
Freshness	15%	Decays 0.2/year from a 2025 baseline, floor 0.2
Cross-filing consistency	15%	1.0 if ≥3 same-company docs corroborate, 0.75 if 2, else 0.4
Retrieval consistency	10%	Computed from `query_variants_hit` (placeholder signal)

5.5 Ingestion Pipeline (`ingestion/`)

`parser.py` `parse_pdf()` (page/block dictionaries), `extract_text_with_positions()` (span-level with bounding boxes), `get_page_text()` – all via PyMuPDF.

`chunker.py` `chunk_document(pages, doc_id, source, target_tokens=400, overlap_tokens=80)`: a heading-aware sliding window. Headings are detected via a font-size heuristic (`span.size > body_size * 1.2`); a new chunk starts at a heading boundary once enough tokens have accumulated, otherwise the window slides with overlap. `chunk_id` is a SHA-256 hash of `(doc_id, page, index, text[:100])`. `_deduplicate()` drops chunks whose first 200 characters hash identically, keeping the first occurrence.

`metadata.py` `detect_section_type()` (regex cascade: audited financials → notes → MD&A → letter → unknown, with a `"note"` sub-check redirecting some audited-financials matches to NOTES), `detect_fiscal_year()` (regex `FY\d{4}`, falling back to a bare-year scan 2024 → 2020), `detect_company()` (filename plus first-200-character text hints for Infosys/TCS/Wipro, handling underscore/hyphen normalization and a "Tata Consultancy" alias), `section_type_confidence_weight()` (weights 1.0 / 0.85 / 0.65 / 0.40 / 0.50 – see note in Table 5.1).

`pipeline.py` `ingest_pdf(pdf_path, doc_id=None, overwrite=True)`: parse → chunk → batch-embed → `ensure_collection()` → optional `delete_by_doc_id()` (for re-ingestion idempotency) → `upsert_chunks()`. `ingest_directory(directory, pattern="*.pdf")`: batch-ingests every matching file, capturing per-file errors without aborting the batch.

Table 5.1: Section-type confidence weights as implemented in code (authoritative) versus as documented in `docs/ARCHITECTURE.md`. The code values govern actual system behavior.

Section type	Code weight	Docs-stated weight
Audited financials	1.00	1.0
Notes	0.85	0.8
MD&A	0.65	0.7
Letter	0.40	0.5
Unknown	0.50	0.4

5.6 Observability (`observability/tracer.py`)

`setup_tracing()` is idempotent (guarded by a `_configured` flag) and configures `structlog` with either a `ConsoleRenderer` (text format) or `JSONRenderer` (JSON / OpenTelemetry-enabled format), merging context variables and adding log level plus ISO timestamp. When `OTEL_ENABLED` is set, an OpenTelemetry `TracerProvider` with an `OTLPSpanExporter` is additionally configured. The `traced(name=None)` decorator logs start/complete/error events with millisecond latency for any decorated async function.

Chapter 6

Data Models

All models are defined in `core/models.py` as `dataclasses` (`from __future__ import annotations`).

6.1 Enumerations

```python
class SectionType(str, Enum):
    AUDITED_FINANCIALS = "audited_financials"
    MDA = "mda"
    NOTES = "notes"
    LETTER = "letter"
    UNKNOWN = "unknown"

class AuditStatus(str, Enum):
    VERIFIED = "verified"
    UNCERTAIN = "uncertain"
    UNVERIFIABLE = "unverifiable"
```

6.2 Core Dataclasses

```python
@dataclass
class Citation:
    document: str
    page: int
    snippet: str
    claim: str
    confidence: float
    section_type: str
    # to_dict()

@dataclass
class Chunk:
    chunk_id: str
    doc_id: str
    source: str
    text: str
    page: int
    section: str
    section_type: SectionType
    token_count: int
```

```
    fiscal_year: str = ""
    company: str = ""
    # to_payload() / from_payload() -- Qdrant payload (de)
        serialization

@dataclass
class RankedChunk:
    chunk: Chunk
    retrieval_score: float
    confidence_score: float
    # to_citation(claim="") -> Citation

@dataclass
class AuditedClaim:
    claim: str
    citation: Citation
    audit_status: AuditStatus
    audit_reason: str
    # to_dict()

@dataclass
class SubtaskResult:
    subtask: str
    ranked_chunks: list[RankedChunk]
    kpis: list[dict]
    claims: list[AuditedClaim]
    agents_used: list[str] = field(default_factory=list)

@dataclass
class AuditLog:
    task_id: str
    timestamp: str
    user_query: str
    plan: list[str]
    retrievals: dict[str, list[str]]
    claims: list[dict]
    flagged_uncertain: list[str]
    blocked_unverifiable: list[str]
    agents_invoked: list[str]
    latency_ms: int
    user_id: str = "anonymous"
    session_id: str = ""
    # to_dict()

@dataclass
class MemoryRecord:
    memory_id: str
    user_id: str
    session_id: str
    task_id: str
    text: str
    timestamp: str
    # to_payload() / from_payload() -- Qdrant point payload, not
        to_dict()

@dataclass
class AnalysisReport:
    task_id: str
```

```
    query: str
    summary: str
    verified_claims: list[AuditedClaim]
    uncertain_claims: list[AuditedClaim]
    audit_log: AuditLog
    # to_dict()
```

6.3 Qdrant Collection Schema

Collection `finsight_chunks`: 384-dimensional vectors (`all-MiniLM-L6-v2`), cosine distance. Payload fields: `chunk_id`, `doc_id`, `source`, `text`, `page`, `section`, `section_type`, `token_count`, `fiscal_year`, `company`. Payload-level filtering is used for `company` during dense search directly in Qdrant; fiscal-year filtering is instead performed in-memory over the BM25 payload set, since Qdrant's filter requires exact string matches and fiscal-year expressions need substring/`"FY"`-prefix handling.

Collection `finsight_memory` (`settings.memory_collection`): same 384-dimensional / cosine configuration, reusing `retrieval/embedder.py`. Payload fields: `memory_id`, `user_id`, `session_id`, `task_id`, `text`, `timestamp`. Short-term recall filters on `session_id` (exact match, then sorted by `timestamp` in Python); long-term recall filters on `user_id` and does a genuine dense vector search against the embedded current query.

6.4 Audit Log Example

The following excerpt (structurally faithful to a real audit log, `audit_logs/0a0601db-...json`, for the query *"revenue per employee of all the companies"*) illustrates the schema in practice:

```
{
  "task_id": "0a0601db-...",
  "timestamp": "2026-...",
  "user_query": "revenue per employee of all the companies",
  "plan": ["...", "..."],
  "retrievals": {"subtask 1": ["chunk_id_a", "chunk_id_b"]},
  "claims": [ { "claim": "...", "citation": {...},
                "audit_status": "uncertain",
                "audit_reason": "..." } ],
  "flagged_uncertain": ["..."],
  "blocked_unverifiable": [],
  "agents_invoked": ["planner", "retriever", "analyst",
                     "auditor", "comparator", "synthesizer"],
  "latency_ms": 50466
}
```

Notably, in this real run the AuditorAgent correctly classified the derived-metric claim "revenue per employee for Infosys ≈ Rs. 453.1 lakh" as `uncertain` – the supporting snippet confirms revenue but not headcount, so the ratio cannot be verified from the cited evidence – rather than either fabricating verification or silently accepting the LLM's arithmetic. The Planner also over-expanded this query to include companies absent from the corpus (HCL, Cognizant); those subtasks correctly retrieved no evidence and produced no spurious verified claims, evidencing that the hard-blocking design works as intended under realistic query drift.

Chapter 7

API Reference

7.1 Endpoint Summary

Route	Method	Handler file	Description
`/query`	POST	`api/routes/query.py`	Blocking full pipeline. Body: `{query, company_filter?, fiscal_year_filter?, confidence_threshold?, session_id?}`. Returns `AnalysisReport.to_dict()` (whose `audit_log` carries the `session_id` to reuse for a follow-up).
`/query/stream`	POST	`api/routes/query_stream.py`	Server-Sent-Events streaming variant, same request body; events: `start, memory_recalled, planned, retrieved, analyzed, audited, compared, remembered, done, error`.
`/ingest/upload`	POST	`api/routes/ingest.py`	Multipart PDF upload (`file=@x.pdf`), optional `doc_id`, `overwrite` (default true). Triggers `ingest_pdf()`.
`/eval/collection`	GET	`api/routes/eval.py`	Qdrant collection stats (name, vector/point counts, status).
`/eval/audit-logs`	GET	`api/routes/eval.py`	Lists up to 20 most recent audit log filenames plus total count.
`/eval/audit-logs/{log_id}`	GET	`api/routes/eval.py`	Returns a specific audit log JSON (tolerant of a missing `.json` suffix).
`/sessions/{id}`	GET, DELETE	`api/routes/sessions.py`	The caller's own turns for that session (404, not silent filtering, if it belongs to another user); delete clears it.
`/memory`	GET, DELETE	`api/routes/memory.py`	The caller's own long-term memory records; delete wipes them.
`/metrics/json`	GET	`api/routes/metrics.py`	Full metrics snapshot.

Route	Method	Handler file	Description
`/health`	GET	`api/main.py`	Liveness probe: `{"status": "ok", "model": <groq_model>}`.
`/ui`, `/ui/`	GET	`api/main.py`	Serves the web UI (no-cache headers); static files mounted beneath.
`/dashboard`	GET	`api/main.py`	Serves the metrics dashboard.
`/`	GET	`api/main.py`	Redirects to `/ui`.

All routes except `/health`, `/ui*`, `/dashboard`, and `/docs` are protected by `AuthMiddleware` (`Authorization: Bearer <key>` → `user_id`, active only when `API_KEYS` is configured – see §2.5) and `GuardrailsMiddleware` (POST/PUT bodies scanned for prompt-injection patterns); all routes sit behind `CORSMiddleware` (`allow_origins=["*"]`). Interactive Swagger documentation is auto-served by FastAPI at `/docs`.

7.2 Streaming Query Route

`api/routes/query_stream.py` defines the SSE variant of the pipeline. It emits an ordered sequence of named events as each stage completes:

`start` → `memory_recalled` → `planned` → `retrieved`* → `analyzed`* → `audited` → `compared` → `remembered` → `done` (or `error` at any point)

(*emitted once per subtask). Both routes drive the same compiled `StateGraph` through `orchestration/runner.py`; the streaming route passes an `on_event` callback that forwards each custom-stream event into the SSE response via an `asyncio.Queue`.

7.3 Shared Route Helpers

`api/routes/query.py` defines the `QueryRequest` Pydantic model (`query`, `company_filter`, `fiscal_year_filter`, `confidence_threshold`, `session_id`) and `_save_audit_log()` (writes to `settings.audit_log_dir/<task_id>`), reused by both routes. `user_id` is read from `request.state.user_id` (set by `AuthMiddleware`), never from the request body, so a caller cannot spoof another user's identity. Subtask parsing and audit-result deserialisation – route-level helpers in the Semantic Kernel era, reparsing JSON strings – no longer exist as separate functions: `orchestration.graph.FinSightState` carries typed values end to end.

7.4 Guardrails Middleware

`GuardrailsMiddleware` (`api/middleware/guardrails.py`) is implemented as **pure ASGI** middleware rather than `BaseHTTPMiddleware`. It buffers the full POST/PUT request body and scans it against a set of prompt-injection patterns (instructions to ignore/forget prior context, "you are now", "jailbreak", `<script>` tags, `[system]` markers, etc.), rejecting matches with an HTTP 400 JSON response. When no match is found, the buffered body is replayed to the first downstream `receive()` call, and subsequent calls delegate to the real `receive`. This specific replay design exists to remain compatible with SSE streaming responses, which `BaseHTTPMiddleware` cannot support without buffering the entire response.

7.5 Auth Middleware

`AuthMiddleware` (`api/middleware/auth.py`) is also pure ASGI, but header-only – it never buffers the request body, so it needs none of `GuardrailsMiddleware`'s receive-patching. It reads `Authorization: Bearer <key>` from `scope["headers"]` and resolves it against `settings.api_keys` (parsed as `"key:user_id,..."`). Empty `API_KEYS` (the default) skips straight to `user_id="anonymous"`; configured, a missing or unknown key gets a `401 JSONResponse` before any handler runs, on every path except `/health`, `/ui*`, `/dashboard`, `/docs`, and `/openapi.json`. Registered after `GuardrailsMiddleware` in `api/main.py` so it executes *first* (Starlette runs middleware in reverse add-order) – a bad key is rejected before the body is buffered and scanned.

7.6 Metrics Store

`MetricsStore` (`api/metrics_store.py`) is a GIL-protected in-process singleton (module-level instance `metrics`) tracking:

- Total queries, errors, and in-flight request counts.
- A rolling 200-sample latency deque, exposing p50/p95/average/min/max plus fixed latency buckets.
- Per-agent rolling 100-sample latency deques for all six agent stages.
- A 50-entry recent-queries ring buffer and a 20-entry recent-errors ring buffer.
- Running totals of verified / uncertain / blocked claims.

Its `snapshot()` method produces the full JSON body served at `/metrics/json`.

Chapter 8

Configuration

8.1 Environment Variables

`core/config.py::Settings` is a `pydantic-settings` model (`extra="ignore"`) reading `.env`. `.env.example` mirrors this table exactly.

Variable	Default	Purpose
`groq_api_key`	`""`	Groq API key (required in practice)
`groq_model`	`llama-3.3-70b-versatile`	Primary LLM
`groq_base_url`	`https://api.groq.com/openai/v1`	OpenAI-compatible endpoint
`fallback_api_key`	`""`	Reserve LLM API key
`fallback_model`	`""`	Reserve LLM model name
`fallback_base_url`	`""`	Reserve LLM endpoint (any OpenAI-compatible service)
`qdrant_host`	`localhost` (`qdrant` in Docker)	Qdrant host
`qdrant_port`	`6333`	Qdrant HTTP port
`qdrant_collection`	`finsight_chunks`	Collection name
`embedding_model`	`sentence-transformers/all-MiniLM-L6-v2`	Bi-encoder
`reranker_model`	`cross-encoder/ms-marco-MiniLM-L6-v2`	Cross-encoder
`chunk_size`	`400`	Target chunk size (tokens)
`chunk_overlap`	`80`	Overlap (tokens)
`retrieval_top_k`	`20`	Candidates per retriever
`rerank_top_k`	`5`	Final chunks after reranking
`confidence_threshold`	`0.65`	VERIFIED cutoff
`hallucination_fallback_threshold`	`0.50`	UNCERTAIN floor
`audit_log_dir`	`audit_logs`	Per-run audit log output directory
`api_keys`	`""`	Comma-separated `key:user_id` pairs. Empty disables auth – every caller is `user_id="anonymous"`
`memory_collection`	`finsight_memory`	Qdrant collection for `MemoryRecord`s
`memory_enabled`	`true`	Kill switch – `false` skips `recall_memory`/`remember` entirely

Variable	Default	Purpose
`session_max_turns`	`6`	Max short-term turns recalled per session
`long_term_top_k`	`3`	Max long-term records recalled per query
`otel_enabled`	`false`	Enable OpenTelemetry export
`otel_endpoint`	`http://localhost:4317`	OTLP gRPC endpoint
`log_level`	`INFO`	Log verbosity
`log_format`	`text`	`text` (console) or `json` (structured)

Docker Compose overrides `QDRANT_HOST=qdrant`, `LOG_FORMAT=json`, and `LOG_LEVEL=INFO` for the `api` service, loading secrets via `env_file: .env`.

Chapter 9

Testing and Evaluation

9.1 Unit Test Suite

Framework: **pytest** + **pytest-asyncio** (`asyncio_mode = "auto"`), `testpaths = ["tests"]`. All tests are pure unit tests of deterministic logic with no live dependency on Qdrant or Groq.

134 tests total.

`test_chunker.py` `_make_chunk_id()` determinism/uniqueness; `_deduplicate()` behavior (exact-duplicate removal, order preservation, 200-character-prefix truncation edge case).

`test_confidence.py` `_freshness_score()`, `_cross_filing_score()`, `compute_confidence()` (bounds, section-type ordering, rerank-score precedence), `build_ranked_chunks()`.

`test_ingestion.py` `detect_fiscal_year()`, `detect_section_type()` (financials/MD&A/letter/notes/unknow `detect_company()` (including the TCS alias).

`test_models.py` Dataclass round-trips: `Citation.to_dict()`, `Chunk.to_payload()`/`from_payload()`, `AuditedClaim` for every `AuditStatus`, `MemoryRecord` round-trip, `AuditLog` `user_id`/`session_id` defaults.

`test_retrieval.py` `_tokenize()`, `BM25Retriever.search()`, `reciprocal_rank_fusion()` (including an exact RRF score assertion of 2/61 and rank-1-vs-rank-2 tie behavior).

`test_unit_normalizer.py` 30+ cases covering `normalize_text()` / `normalize_subtask_results()`: USD billion/million/thousand/trillion/bn conversions, INR crore/lakh/million/billion conversions, idempotency on already-converted labels, non-mutation of input dicts, and edge cases (zero, decimals, Indian comma grouping, mixed currencies within one sentence).

`test_graph.py` `StateGraph` construction (all seven nodes present, including `recall_memory`/`remember`), `fan_out` `Send` shape, a stubbed end-to-end run through `build_graph()`, the auditor-abort / comparator-degrade error paths, and memory-specific cases: recalled context reaches `plan_task` and nowhere else, a broken memory service degrades to no context without failing the run, `memory_enabled=False` skips recall entirely.

`test_memory.py` `build_turn_text()` / `format_memory_context()` as pure functions – no Qdrant.

`test_auth.py` `AuthMiddleware` – key parsing, bearer-token extraction, anonymous pass-through when unconfigured, 401 on missing/bad key when configured, exempt-path bypass. First test in the suite to exercise ASGI middleware directly, via `httpx.ASGITransport`.

No integration tests exercise the live FastAPI app, Qdrant, or Groq directly within `tests/` – `test_auth.py` is the one exception, wrapping a minimal Starlette app (not the full `create_app()`)

purely to drive real ASGI request/response plumbing without pulling in Qdrant/embedding-model imports. End-to-end exercise of the live app is otherwise the evaluation harness's role, described below.

9.2 Evaluation Harness

`evaluation/harness.py::run_harness(query_file)` is an end-to-end scoring tool distinct from the pytest suite: it drives the *live* `/query` endpoint (`http://localhost:8000/query`, 360 s timeout) rather than mocking it. Each test case carries an `expected_behavior` label, and `score_result()` applies behavior-specific pass/fail logic:

`expected_behavior`	**Passes when**
`unverifiable`	All claims blocked / no verified claims pro
`unverifiable_or_insufficient_evidence`	Blocked, or insufficient-evidence response
`uncertain_or_unverifiable`	Claims are uncertain or blocked (none fals
`verified_or_uncertain` / `partial_or_insufficient_evidence` (default)	At least one verified claim, or an explicit p Requires $\geq$1 verified claim

Each run writes a timestamped `harness_<epoch>.json` summary to `evaluation/results/`.

9.3 Query Sets

`evaluation/queries/happy_path.json` 4 cases targeting verifiable facts: Infosys margin, Infosys/TCS revenue-growth comparison, Wipro revenue drivers, TCS headcount – all `expected_behavior: verified_or_uncertain`.

`evaluation/queries/adversarial.json` 4 hallucination-eliciting cases: a future-year query (FY2030, no data can exist), a fabricated event ("what did the CEO say about acquiring Google"), a cross-domain query (Microsoft/Apple, absent from the corpus), and a numerically precise but implausibly-stated figure (exact free cash flow to the rupee).

9.4 Recorded Results

`evaluation/results/` accumulates timestamped run artifacts (git-ignored). A representative happy-path run reports a **4/4 pass rate (1.0)**, with per-test latency ranging roughly 8–30 s – e.g. the cross-company revenue-growth comparison query returns close to 20 verified claims. The project README states currently published scores of **4/4 on the happy-path set and 4/4 on the adversarial set**; the previous 3/4 was a timeout on a five-company cross-domain query, resolved once the audit/compare superstep (§2.5's parent chapter) removed the sequential-latency cause. Re-verified against this identity/memory change: still 4/4 / 4/4, since recalled memory only ever reaches the Planner prompt and cannot affect claim verification.

Chapter 10

Build, Deployment, and CI

10.1 Dockerfile

A two-stage build:

1. **Builder stage** (`python:3.11-slim`): installs `uv`, extracts runtime dependencies from `pyproject.toml` via the standard-library `tomllib`, creates a `.venv`, and installs dependencies into it. This layer only rebuilds when `pyproject.toml` or `uv.lock` change.
2. **Runtime stage** (`python:3.11-slim`): installs only `curl` (used by the healthcheck), copies the pre-built `.venv` from the builder stage (no pip/uv/build toolchain present in the final image), copies application source (respecting `.dockerignore`), creates `audit_logs/` and `data/filings/`, and runs as a non-root user `finsight`. Exposes port 8000 and defines a `HEALTHCHECK` against `/health` with a 60 s start period (to accommodate cold-start model downloads). Entrypoint: `uvicorn api.main:app -workers 1 -access-log -log-level warning` – a single worker is deliberate, preserving the in-process SK kernel singleton and metrics store across requests.

10.2 Docker Compose

Two services:

`qdrant` Image `qdrant/qdrant:latest`; ports 6333/6334; volume-mounts `./qdrant_storage`; TCP healthcheck; `json-file` logging (20 MB × 3 files).

`api` Built from the local `Dockerfile`; port 8000; environment overrides for Docker networking (`QDRANT_HOST=qdrant`, `LOG_FORMAT=json`); secrets loaded via `env_file: .env`; volume-mounts `./audit_logs` and `./data`; `depends_on: qdrant: condition: service_healthy`; `json-file` logging (50 MB × 5 files); its own HTTP healthcheck.

10.3 Continuous Integration

`.github/workflows/ci.yml` (GitHub Actions), triggered on push/PR to `main`: sets up Python 3.11 and `uv`, runs `uv sync -group dev`, then `ruff check .`, `ruff format -check .`, and finally `pytest tests/ -v -tb=short`. No separate Docker build/publish or continuous-deployment job is present.

10.4 Local Development Workflow

1. `docker compose up qdrant -d` – start only the vector database.
2. `uv sync` – install dependencies locally.
3. `uvicorn api.main:app -reload` – run the API with hot reload outside Docker for fast iteration.

`.dockerignore` excludes `.env`, `qdrant_storage`, `tests`, and `__pycache__` (among others) from the Docker build context and final image.

Chapter 11

Supporting Scripts

11.1 scripts/download_filings.py

A multi-strategy downloader for the seed corpus of annual-report PDFs, attempting each strategy in order until one succeeds:

1. The BSE India filing API, queried by scrip code.
2. A table of known direct PDF URL hints.
3. HTML scraping of investor-relations pages for `.pdf` links matching the target fiscal year.
4. An interactive browser fallback that opens the page and waits for a manual save, for filings that cannot be located programmatically.

Downloads are validated against a minimum file size and expected content type, and a final summary of successes/failures is reported.

11.2 main.py

A trivial placeholder entrypoint (`print("Hello from finsight!")`) retained from project scaffolding; it is *not* the real application entrypoint. The actual service is served via `uvicorn api.main:app`.

Chapter 12

Design Decisions and Known Deviations

This chapter records intentional design tradeoffs and points where implementation details diverge from the idealized descriptions in `docs/ARCHITECTURE.md`, as verified directly against the source.

1. **LangGraph replaced Semantic Kernel** (`docs/DECISIONS.md` Decision 11, 2026-09-09). SK's dispatch layer – `kernel.invoke()` at every hop, citations threaded as JSON strings inside `KernelArguments` – was reversed once eighteen months of living with the code showed the plan's shape never actually varies: plan → (retrieve + analyse per subtask, parallel) → audit ‖ compare → synthesise is a fixed topology with a dynamic fan-out width, which is exactly what LangGraph's `Send` API is for. Retriever, Analyst, Auditor, and Comparator are now plain `async` functions called directly from graph nodes; Planner and Synthesizer remain prompt-only roles dispatched via `chat_completion()` / `chat_completion_hedged()`, unchanged by the migration.
2. **Auditor/Comparator concurrency is now structural, not an implementation-level optimization layered on top of a sequential diagram.** They are one LangGraph superstep – two nodes with the same incoming edge and no edge between them – so the concurrency holds on *both* the blocking and streaming routes, not only the streaming route as before the LangGraph migration.
3. **Per-user identity and short/long-term memory** (`docs/DECISIONS.md` Decision 12, 2026-09-09). Simple `API_KEYS` → `user_id` mapping (not full accounts/RBAC) scopes a new Qdrant collection, `finsight_memory`, recalled by two best-effort graph nodes bracketing the pipeline. The load-bearing constraint: recalled memory reaches `PLANNER_PROMPT` only, never `SYNTHESIZER_PROMPT` – it can bias what gets searched for, never what gets asserted as a claim, so a user's own unreviewed history can never become an unverified figure in the report. No LLM-based memory consolidation step exists, deliberately: each `MemoryRecord` is exactly what the pipeline already produced (query + plan + summary excerpt), traceable to a real `task_id`, never a new synthesis of it.
4. **Section-type confidence weights in code differ slightly from documented values** (Table 5.1); the code is authoritative for actual system behavior.
5. **All currency/unit arithmetic is deterministic**, performed in `core/unit_normalizer.py` before either the Comparator or Synthesizer LLM stage sees the data – explicitly to prevent LLM arithmetic hallucination on financial figures.
6. **Hard blocking over soft flagging.** The Auditor's UNVERIFIABLE classification removes a claim from the synthesized report entirely; it is never surfaced to the end user with a disclaimer, unlike the UNCERTAIN tier.

7. `retrieval/chunker.py` **and** `retrieval/dummy_test.py` **are non-canonical.** They duplicate ingestion logic for manual/ad hoc PDF experimentation during development and are not reachable from the production ingestion pipeline (`ingestion/pipeline.py`).